

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	09 - 24 - 26	DATE SUBMITTED:	09 - 24 - 26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

Midterm Exam	Midterm Skills Exam: Ansible Playbook for installing Prometheus
1. PROCEDURE OUTPUT	
1. Create a playbook that installs Prometheus in both Ubuntu and CentOS. Apply the concept of creating roles. 2. Describe how you did step 1. (Provide screenshots and explanations in your report. Make your report detailed such that it will look like a manual.) 3. Show an output of the installed Prometheus for both Ubuntu and CentOS. 4. Make sure to create a new repository in GitHub for this activity.	
Task 0: Initialize Prometheus Role 1. Create Prometheus role 2. Define Defaults 3. Define Handlers 4. Define Config Template <pre>workstation@workstation2:~/CPE212-Santiago/ansible-lab\$ tree roles/prometheus roles/prometheus ├── defaults │ └── main.yml ├── handlers │ └── main.yml ├── meta │ └── main.yml ├── README.md ├── tasks │ └── main.yml ├── templates │ └── prometheus.service.j2 ├── tests │ ├── inventory │ └── test.yml └── vars └── main.yml</pre>	

```
workstation@workstation2:~/CPE212-Santiago/ansible-lab/roles/prometheus/defaults$ cat *
---
# defaults file for prometheus

prometheus_version: "2.52.0"
prometheus_user: "prometheus"
prometheus_group: "prometheus"
prometheus_config_dir: "/etc/prometheus"
prometheus_db_dir: "/var/lib/prometheus"
prometheus_bin_dir: "/usr/local/bin"
```

```
workstation@workstation2:~/CPE212-Santiago/ansible-lab/roles/prometheus/handlers$ cat *
---
# handlers file for prometheus

# [Ubuntu/CentOS]: sudo systemctl daemon-reload
- name: Reload systemd
  ansible.builtin.systemd:
    daemon_reload: true

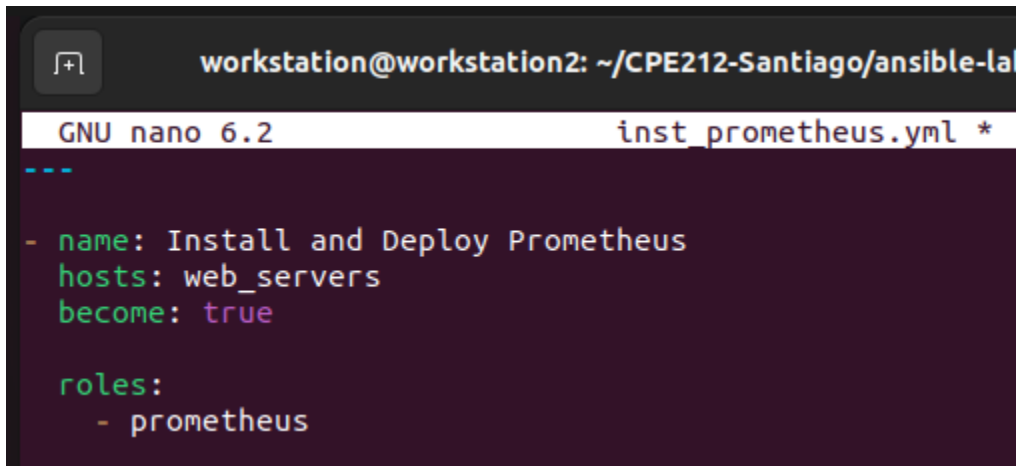
# [Ubuntu/CentOS]: sudo systemctl restart prometheus
- name: Restart prometheus
  ansible.builtin.systemd:
    name: prometheus
    state: restarted
```

```
workstation@workstation2:~/CPE212-Santiago/ansible-lab/roles/prometheus/templates$ cat *
[Unit]
Description=Prometheus Monitoring System
Wants=network-online.target
After=network-online.target

[Service]
User={{ prometheus_user }}
Group={{ prometheus_group }}
Type=simple
Restart=on-failure
RestartSec=5s
ExecStart={{ prometheus_bin_dir }}/prometheus \
  --config.file={{ prometheus_config_dir }}/prometheus.yml \
  --storage.tsdb.path={{ prometheus_db_dir }} \
  --web.console.templates={{ prometheus_config_dir }}/consoles \
  --web.console.libraries={{ prometheus_config_dir }}/console_libraries \
  --web.listen-address=0.0.0.0:9090 \
  --web.enable-lifecycle \
  --log.level=info

[Install]
WantedBy=multi-user.target
```

Install prometheus.yml file to initiate installation:

A screenshot of a terminal window. The title bar shows a window icon, a plus sign, and the text 'workstation@workstation2: ~/CPE212-Santiago/ansible-lab'. The terminal content shows the GNU nano 6.2 editor editing 'inst_prometheus.yml'. The file content is as follows:

```
---  
  
- name: Install and Deploy Prometheus  
  hosts: web_servers  
  become: true  
  
  roles:  
    - prometheus
```

Task 1: Prepare the Server in prometheus/tasks/main.yml

List the commands required to:

1. Update the package repository.
2. Upgrade installed packages.
3. Install the utilities required by the installation.

OUTPUT

```

---
# PRE-TASKS: UPDATE AND UPGRADE
- name: Install and Deploy Prometheus
  hosts: web_servers
  become: true

  roles:
    - prometheus

# [CentOS]: sudo dnf check-update && sudo dnf upgrade -y
- name: Update repository cache and upgrade packages (CentOS / RHEL)
  tags: always
  ansible.builtin.dnf:
    name: "*"
    state: latest
    update_cache: true
    update_only: true
  when: ansible_os_family == "RedHat"

# [Ubuntu]: sudo apt update && sudo apt dist-upgrade -y
- name: Update repository cache and upgrade packages (Ubuntu / Debian)
  tags: always
  ansible.builtin.apt:
    upgrade: dist
    update_cache: true
  when: ansible_os_family == "Debian"

# [Ubuntu]: sudo apt install -y wget tar curl
# [CentOS]: sudo dnf install -y wget tar curl

- name: Install utilities (wget, tar, curl)
  ansible.builtin.package:
    name:
      - wget
      - tar
      - curl
    state: present

```

Task 2: Create the Prometheus User

Identify the commands required to:

1. Create a dedicated `prometheus` system user.
`sudo useradd --no-create-home prometheus`
2. Prevent the user from having a normal login shell.

```
sudo usermod -s /bin/false prometheus
```

3. Verify that the user was successfully created.

```
id prometheus
```

OUTPUT

```
# TASK 2: CREATE THE PROMETHEUS USER

# [Ubuntu/CentOS]: sudo groupadd --system prometheus
- name: Create prometheus system group
  ansible.builtin.group:
    name: "[[ prometheus_group ]]"
    state: present
    system: true

# [Ubuntu]: sudo useradd --system --no-create-home --shell /bin/false -g prometheus prometheus
# [CentOS]: sudo useradd --system --no-create-home --shell /sbin/nologin -g prometheus prometheus
# [Verification]: id prometheus
- name: Create prometheus system user with disabled login shell
  ansible.builtin.user:
    name: "[[ prometheus_user ]]"
    group: "[[ prometheus_group ]]"
    shell: "[[ '/bin/false' if ansible_os_family == 'Debian' else '/sbin/nologin' ]]"
    create_home: false
    system: true
    state: present
```

Part 3: Create Prometheus Directories

Identify the commands required to:

1. Create `/etc/prometheus`.
`sudo mkdir /etc/prometheus`
2. Create `/var/lib/prometheus`.
`sudo mkdir /var/lib/prometheus`
3. Assign the appropriate owner and group.
`sudo chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus`
4. Verify the directory permissions and ownership.
`ls -ld /etc/prometheus /var/lib/prometheus`

OUTPUT

```
# TASK 3: CREATE PROMETHEUS DIRECTORIES

# [Ubuntu/CentOS]: sudo mkdir -p /etc/prometheus
- name: Create configuration directory /etc/prometheus
  ansible.builtin.file:
    path: "{{ prometheus_config_dir }}"
    state: directory
    owner: "{{ prometheus_user }}"
    group: "{{ prometheus_group }}"
    mode: '0755'

# [Ubuntu/CentOS]: sudo mkdir -p /var/lib/prometheus
# [Verification]: ls -ld /etc/prometheus /var/lib/prometheus
- name: Create data directory /var/lib/prometheus
  ansible.builtin.file:
    path: "{{ prometheus_db_dir }}"
    state: directory
    owner: "{{ prometheus_user }}"
    group: "{{ prometheus_group }}"
    mode: '0755'
```

Part 4: Download Prometheus

Identify the commands required to:

1. Navigate to a temporary working directory.
`cd /tmp`
2. Download the Prometheus archive.
`wget https://github.com/prometheus/prometheus/releases/download/v2.52.0/prometheus-2.52.0.linux-amd64.tar.gz`
3. Verify that the downloaded file exists.
`ls -lh prometheus-2.52.0.linux-amd64.tar.gz`
4. Extract the Prometheus archive.
`tar -xvf prometheus-2.52.0.linux-amd64.tar.gz`
5. Verify the extracted directory and its contents.
`ls -la prometheus-2.52.0.linux-amd64`

Include the Prometheus version used in your procedure.

OUTPUT

```
# TASK 4: DOWNLOAD PROMETHEUS
# [Ubuntu/CentOS]: cd /tmp
# [Ubuntu/CentOS]: wget https://github.com/prometheus/prometheus/releases/download/v2.52.0/prometheus-2.52.0.linux-amd64.tar.gz
# [Ubuntu/CentOS]: tar -xvf prometheus-2.52.0.linux-amd64.tar.gz
# [Verification]: ls -lh prometheus-2.52.0.linux-amd64.tar.gz && ls -la prometheus-2.52.0.linux-amd64
- name: Download and extract Prometheus archive
  ansible.builtin.unarchive:
    src: "https://github.com/prometheus/prometheus/releases/download/v{{ prometheus_version }}/prometheus-{{ prometheus_version }}.linux-amd64.tar.gz"
    dest: /tmp
    remote_src: true
    creates: "/tmp/prometheus-{{ prometheus_version }}.linux-amd64"
```

Task 5: Install the Prometheus Binaries

Identify the commands required to:

1. Locate the `prometheus` binary.
`ls -l prometheus-2.52.0.linux-amd64/prometheus`
2. Locate the `promtool` binary.
`ls -l prometheus-2.52.0.linux-amd64/promtool`
3. Copy both binaries to `/usr/local/bin`.
`sudo cp prometheus-2.52.0.linux-amd64/prometheus prometheus-2.52.0.linux-amd64/promtool /usr/local/bin/`
4. Set the appropriate ownership.
`sudo chown prometheus:prometheus /usr/local/bin/prometheus /usr/local/bin/promtool`
5. Verify the installed Prometheus version.
`prometheus --version`
6. Verify the installed `promtool` version.
`promtool --version`

OUTPUT

```
# TASK 5: INSTALL PROMETHEUS BINARIES
# [Ubuntu/CentOS]: sudo cp /tmp/prometheus-2.52.0.linux-amd64/prometheus /usr/local/bin/
# [Ubuntu/CentOS]: sudo cp /tmp/prometheus-2.52.0.linux-amd64/promtool /usr/local/bin/
# [Ubuntu/CentOS]: sudo chown prometheus:prometheus /usr/local/bin/prometheus /usr/local/bin/promtool
# [Ubuntu/CentOS]: sudo chmod 0755 /usr/local/bin/prometheus /usr/local/bin/promtool
# [Verification]: /usr/local/bin/prometheus --version && /usr/local/bin/promtool --version
- name: Copy Prometheus binaries to /usr/local/bin
  ansible.builtin.copy:
    src: "/tmp/prometheus-{{ prometheus_version }}.linux-amd64/{{ item }}"
    dest: "{{ prometheus_bin_dir }}/{{ item }}"
    owner: "{{ prometheus_user }}"
    group: "{{ prometheus_group }}"
    mode: '0755'
    remote_src: true
  loop:
    - prometheus
    - promtool
```

Task 6: Install the Prometheus Configuration Files

Identify the commands required to:

1. Copy `prometheus.yml` to `/etc/prometheus`.
`sudo cp prometheus-2.52.0.linux-amd64/prometheus.yml /etc/prometheus/prometheus.yml`
2. Copy the `consoles` directory.
`sudo cp -r prometheus-2.52.0.linux-amd64/consoles /etc/prometheus/`
3. Copy the `console_libraries` directory.
`sudo cp -r prometheus-2.52.0.linux-amd64/console_libraries /etc/prometheus/`
4. Set the appropriate ownership.
`sudo chown -R prometheus:prometheus /etc/prometheus`
5. Verify the files and directories.
`ls -la /etc/prometheus`

OUTPUT

```
# TASK 6: INSTALL PROMETHEUS CONFIG FILES
# [Ubuntu/CentOS]: sudo cp /tmp/prometheus-2.52.0.linux-amd64/prometheus.yml /etc/prometheus/
# [Ubuntu/CentOS]: sudo chown prometheus:prometheus /etc/prometheus/prometheus.yml
- name: Copy default prometheus.yml to /etc/prometheus
  ansible.builtin.copy:
    src: "/tmp/prometheus-{{ prometheus_version }}.linux-amd64/prometheus.yml"
    dest: "{{ prometheus_config_dir }}/prometheus.yml"
    owner: "{{ prometheus_user }}"
    group: "{{ prometheus_group }}"
    mode: '0644'
    remote_src: true

# [Ubuntu/CentOS]: sudo cp -r /tmp/prometheus-2.52.0.linux-amd64/consoles /etc/prometheus/
# [Ubuntu/CentOS]: sudo cp -r /tmp/prometheus-2.52.0.linux-amd64/console_libraries /etc/prometheus/
- name: Copy consoles and console_libraries directories to /etc/prometheus
  ansible.builtin.copy:
    src: "/tmp/prometheus-{{ prometheus_version }}.linux-amd64/{{ item }}"
    dest: "{{ prometheus_config_dir }}/"
    owner: "{{ prometheus_user }}"
    group: "{{ prometheus_group }}"
    mode: '0755'
    remote_src: true
  loop:
    - consoles
    - console_libraries

# [Ubuntu/CentOS]: sudo chown -R prometheus:prometheus /etc/prometheus
# [Verification]: ls -la /etc/prometheus
- name: Ensure correct recursive ownership on /etc/prometheus
  ansible.builtin.file:
    path: "{{ prometheus_config_dir }}"
    owner: "{{ prometheus_user }}"
    group: "{{ prometheus_group }}"
    recurse: true
```

Task 7: Validate the Configuration

1. Identify the command used to check whether the Prometheus configuration is valid.
`promtool check config /etc/prometheus/prometheus.yml`
2. Explain why configuration validation should be performed before starting the Prometheus service.

This ensures that all of the installed dependencies, file paths, and configured settings are correct in order to guarantee that the Prometheus service will activate and run with no issues.

OUTPUT

```
# TASK 7: VALIDATE THE CONFIGURATION

# [Ubuntu/CentOS]: /usr/local/bin/promtool check config /etc/prometheus/prometheus.yml
- name: Validate Prometheus Configuration
  ansible.builtin.command:
    cmd: "[{ prometheus_bin_dir }]/promtool check config [{ prometheus_config_dir }]/prometheus.yml"
  changed_when: false
```

Task 8: Create the systemd Service

Identify the procedure for creating:

`/etc/systemd/system/prometheus.service` Your documentation must include the complete service file.

Steps:

```
sudo nano /etc/systemd/system/prometheus.service
```

```
[Unit]
Description=Prometheus Monitoring System
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus \
  --web.console.templates=/etc/prometheus/consoles \
  --web.console.libraries=/etc/prometheus/console_libraries

Restart=always

[Install]
WantedBy=multi-user.target
```

Explain the purpose of the important directives, including:

- `User`
This defines the system user account to execute the prometheus service

- `Group`
This specifies the system group owning the running process.
- `ExecStart`
This specifies the binary path and startup flags
- `--config.file`
This locates the main YAML configuration file
- `--storage.tsdb.path`
This defines the directory where Prometheus saves important database metrics
- `Restart`
Serves as a failsafe to restart the daemon if it crashes
- `WantedBy`
This specifies the target run-level, which in this case, is multi.user, which allows it to auto-start during boot up under any user account.

OUTPUT

```
GNU nano 6.2                                prometheus.service.j2
[Unit]
Description=Prometheus Monitoring System
Wants=network-online.target
After=network-online.target

[Service]
User={{ prometheus_user }}
Group={{ prometheus_group }}
Type=simple
Restart=on-failure
RestartSec=5s
ExecStart={{ prometheus_bin_dir }}/prometheus \
  --config.file={{ prometheus_config_dir }}/prometheus.yml \
  --storage.tsdb.path={{ prometheus_db_dir }} \
  --web.console.templates={{ prometheus_config_dir }}/consoles \
  --web.console.libraries={{ prometheus_config_dir }}/console_libraries \
  --web.listen-address=0.0.0.0:9090 \
  --web.enable-lifecycle \
  --log.level=info

[Install]
WantedBy=multi-user.target
```

TASK 8: CREATE SYSTEMD SERVICE

```
- name: Create /etc/systemd/system/prometheus.service
  ansible.builtin.template:
    src: prometheus.service.j2
    dest: /etc/systemd/system/prometheus.service
    owner: root
    group: root
    mode: '0644'
  notify:
    - Reload systemd
    - Restart prometheus
```

Task 9: Start and Enable Prometheus

Identify the commands required to:

1. Reload systemd.
`sudo systemctl daemon-reload`
2. Enable Prometheus to start automatically during boot.
`sudo systemctl enable prometheus`
3. Start Prometheus.
`sudo systemctl start prometheus`
4. Check the service status.
`sudo systemctl status prometheus`

OUTPUT

```
# TASK 9: START AND ENABLE PROMETHEUS
- name: Enable and start Prometheus service
  ansible.builtin.systemd:
    name: prometheus
    enabled: true
    state: started
```

Task 10: Verify Prometheus

Identify the commands required to verify that Prometheus is running correctly. Your

procedure must include:

1. Checking whether port 9090 is listening.
`ss -tulpn | grep 9090`
2. Testing Prometheus locally.
`curl http://localhost:9090`
3. Checking the Prometheus readiness endpoint.
`curl -s http://localhost:9090/-/ready`
4. Checking the Prometheus version.
`prometheus --version`
5. Checking the systemd service status.
`sudo systemctl status prometheus`

OUTPUT

```
# TASK 10: VERIFY PROMETHEUS

# [Ubuntu/CentOS]: ss -tulpn | grep 9090
- name: Verify Prometheus Service is listening on port 9090
  ansible.builtin.wait_for:
    port: 9090
    timeout: 10
```

Part 11: Access Prometheus Through a Browser

Identify how Prometheus can be accessed from another computer. Document the URL format:

```
http://<SERVER-IP>:9090
```

Explain:

1. What <SERVER-IP> represents.
This is the local machine's IP address running Prometheus, which can either be 127.0.0.1/localhost or the actual IP address.
2. Why port 9090 is used.
This is the default network port reserved by Prometheus to allow HTTP communication which also serves as its webpage interface.

3. What should appear when Prometheus is successfully accessed.
The output should be the Prometheus Web User Interface containing different navigation tabs, expression search bar, and customizable ways to visualize system metrics. It should look like the interface below:

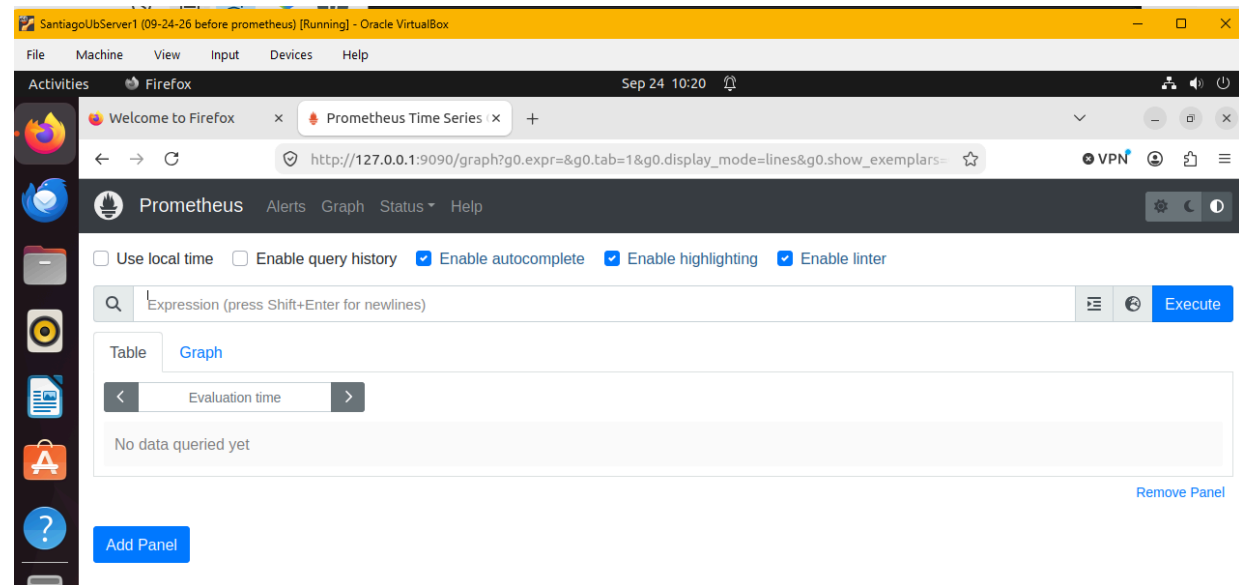
OUTPUT

```
TASK [prometheus : Enable and start Prometheus service] *****
ok: [ubuntu1]
ok: [centos1]

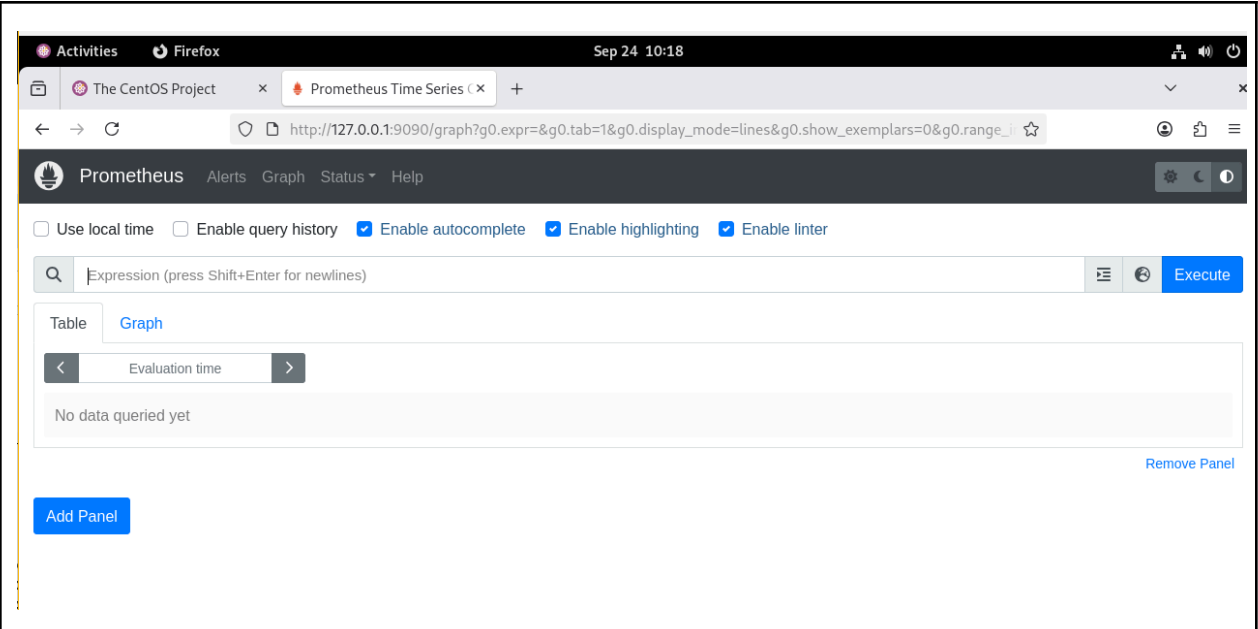
TASK [prometheus : Verify Prometheus Service is listening on port 9090] *****
ok: [ubuntu1]
ok: [centos1]

PLAY RECAP *****
centos1                : ok=14   changed=0    unreachable=0    failed=0    skipped=1    rescued=0
   ignored=0
ubuntu1                : ok=14   changed=0    unreachable=0    failed=0    skipped=1    rescued=0
   ignored=0
workspace@workspace2: ~/CP5212-Santiago/ansible-labs
```

Ubuntu:



CentOS:



DOCUMENTATION STEPS RETRIEVED FROM:
<https://www.akamai.com/cloud/guides/how-to-install-prometheus-and-grafana-on-ubuntu>

GITHUB:
<https://github.com/owen-s-dotdev/CPE212-Santiago.git>

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

4. CONCLUSION/ANALYSIS/LEARNINGS

In conclusion, this exam shows how to install Prometheus on Ubuntu and CentOS systems using ansible automation while implementing Ansible roles.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____
Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____



COMPUTER
ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.